Subject : Programming for Problem Solving Lab.

Unit 3 : Control Statements

Program : B.Tech. - CSE

Course Code : **CSCR 1501**

Semester : 1st

Faculty Teaching : Dr. Shree Harsh Attri

Designation : Associate Professor

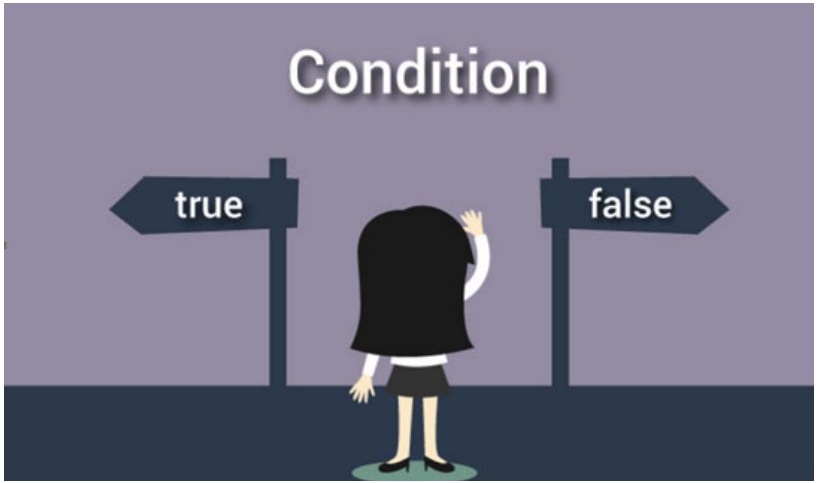
Department : Computer Sc. & Engineering

Sharda University, Greater Noida, U.P

Contents

A. Decisions: if, if-else, Nested if-else, switch case
B. Iterations: while, do while, for loop
C. Execution Sequence: goto, break, continue

Control Statements



Conditional Statements

1. These are also called decision control statements.

2. It allows the programmers to build the programs that determine which statements of the code should be executed and which are to be ignored

Following are the types of decision-making control statements in C language. They are,
a. if statements
b. if else statements
c. nested if statements
d. switch case statements

Conditional Statement

If-else

if

```
if( condition )
{

   //true
}
```

if-else

```
if( condition )
{

   //true
}
else
{

   //false

}
```

if-else if

```
if( condition 1 )
{

   //true
}
else if( condition 2 )
{

   //true
}
else
{

}
```

Nested if

```
if( condition 1 )
{

   if(condition)
     {

     }
   else
     {

     }
}
else
{
  if(condition)
     {

     }
  else
     {

     }
}
```

Switch

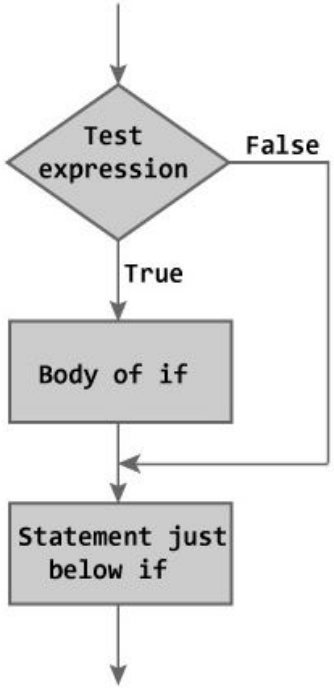
```
switch( expression )
{
    case 1:
    break;

    case 2:
    break;

    case 3:
    break;

    default;

}
```

If statements

The if statement evaluates the test expression inside the parenthesis.

- If the test expression is evaluated to true (nonzero), statements inside the body of if is executed.
- If the test expression is evaluated to false (0), statements inside the body of if are skipped from execution.

```
// Program to display a number if user enters negative number
 // If user enters positive number, that number won't be displayed
  #include <stdio.h>
main()
{
   int number;
   printf("Enter an integer: ");
   scanf("%d", &number);
if (number < 0)        // Test expression is true if number is less than 0
   {
      printf("You entered negative number);
   }
}
```

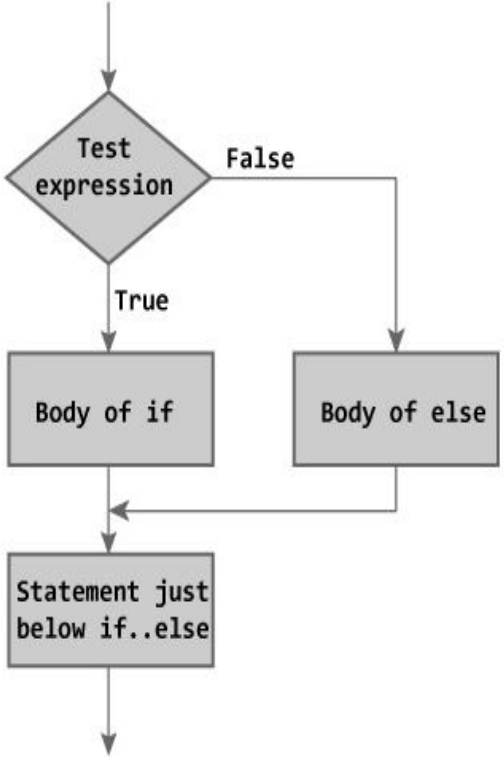


If – else statements

There are two blocks of statements. If condition results true then **if** block gets executed otherwise statements inside **else** block executes.
Note: else cannot exist without if statement.

// Program to check whether an integer entered by the user is odd or even

```
#include <stdio.h>
main()
{
int number;
printf("Enter an integer: ");
scanf("%d",&number);

if( number%2 == 0 ) // True if remainder is 0
     printf("%d is an even integer.",number);
else
     printf("%d is an odd integer.",number);
}
```



Output

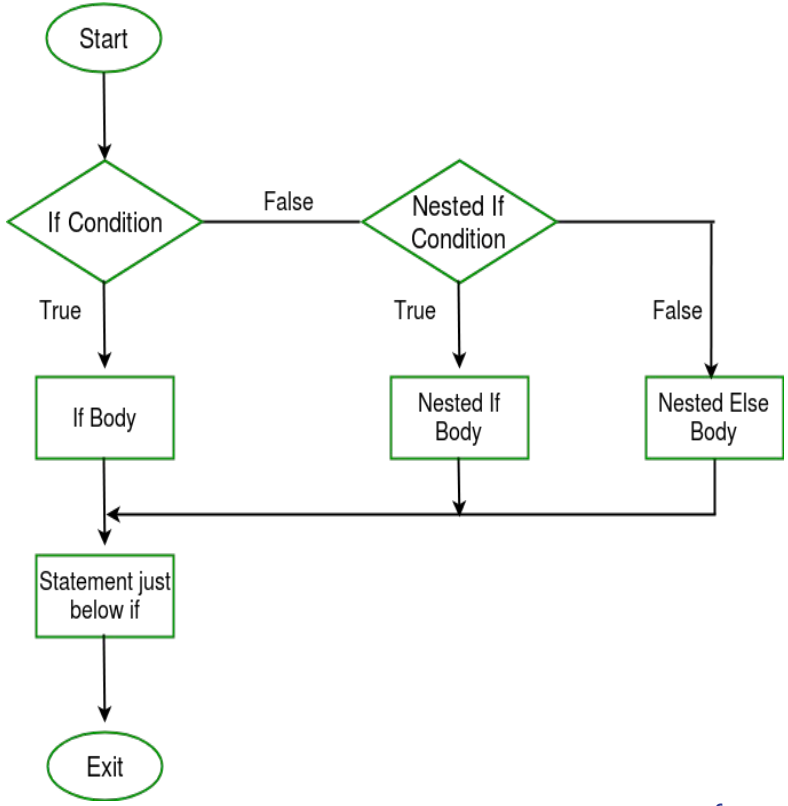
Enter an integer: 7
7 is an odd integer

if - else if - else statements

- The if...else statement executes two different codes depending upon whether the test expression is true or false. Sometimes, a choice has to be made from more than 2 possibilities.

// Program to relate two integers using =, > or <

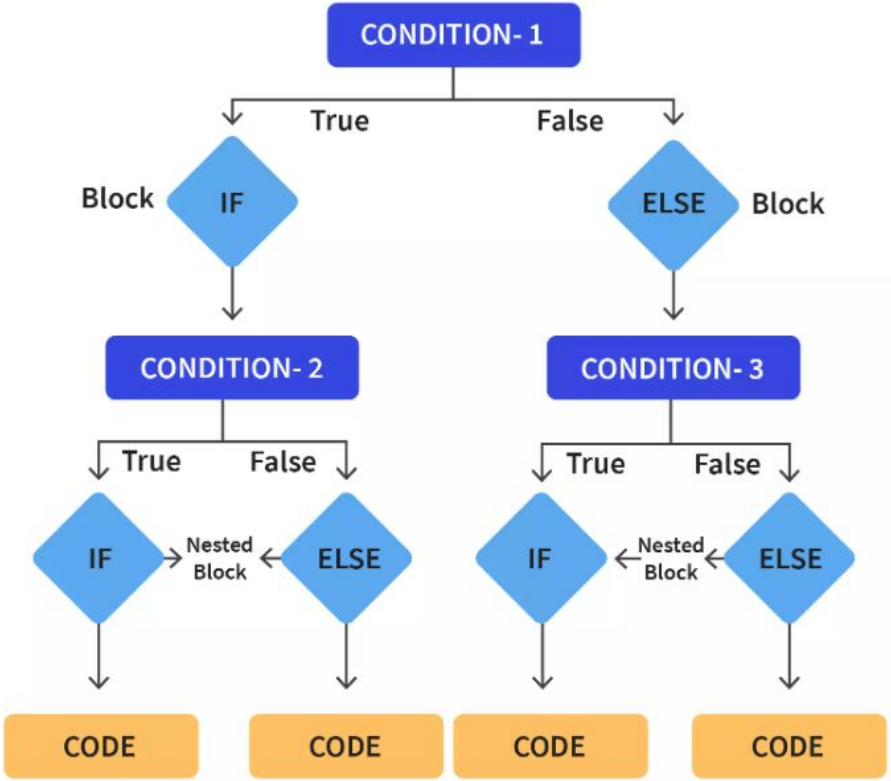
```
#include <stdio.h>
main()
{
int number1, number2;
printf("Enter two integers: ");
scanf("%d %d", &number1, &number2);
if(number1 == number2)  //checks if two integers
        are equal
{
      printf("Result: %d = %d",number1,number2);
}
else if (number1 > number2) //checks if number1 is
            greater than number2
{
      printf("Result: %d > %d", number1, number2);
}
else        // if both test expression is
false
{
      printf("Result: %d < %d",number1, number2);
}
}
```



Nested if else statements

- The nested if...else statement allows you to check for multiple test expressions and execute different codes for more than two conditions.

```
//check if the first condition holds
if (condition 1) {

    //if the second condition holds
    if (condition 2) {
        do something
    }
    //if the second condition does not hold
    else {
        do something else
    }
}
// if the first condition does not hold
else{

    //if the third condition holds
    if (condition 3) {
        do something
    }
    //if the third condition does not hold
    else {
        do something else
    }
}
```



Nested if else statements

Program to find the largest of three numbers using nested if-else

```
#include<stdio.h>
main()
{
      int a,b,c;
      printf("Enter values of a, b and c: ");
      scanf("%d %d %d",&a,&b,&c);
      if(a>b)
      {
            if(a>c)
            printf("%d is largest\n",a);
            else
            printf("%d is largest\n",c);
      }
      else
      {
            if(b>c)
            printf("%d is largest\n",b);
            else
            printf("%d is largest\n",c);
      }
}
```

Exercise

SHARDA UNIVERSITY
Beyond Boundaries

Write a Program

1) To check whether a number is even or odd.

2) To calculate grade based on marks using if-else ladder.

3) To check whether a character is a vowel or consonant.

4) To take four positive integers a, b, c, and d as input. These represent two fractions, the first fraction is **a / c** and the second fraction is **d / b**, find which fraction is larger, using only integer computations (i.e. do not use floating-point operations).

5) To assist in calculating the final payable amount on an e-commerce platform based on the order value and applicable discounts. The discount policy is as follows:
   - 20% discount is applied if the order amount exceeds ₹5000.
   - 10% discount is applied if the order amount exceeds ₹2000 but is less than or equal to ₹5000. \
   - No discount is applied for order amounts of ₹2000 or less.

   The program should prompt the user to enter the total order amount and based on the discount criteria, calculate the discount and deduct it from the total amount of order. Display both the discount applied and the final amount to be paid by the customer. The program should be generating the output for the discount percentage, the discount amount and the final payable amount after applying the discount.

6) To read a patient's body temperature in degrees Fahrenheit and evaluates their health condition based on predefined medical thresholds. The program should classify the body temperature as follows:
   - Below 95°F → Hypothermia
   - 95°F to less than 99°F → Normal
   - 99°F to less than or equal to 102°F → Mild Fever
   - above 102°F → High Fever

8) To take as input the coefficients of two straight lines in the general form:

   Line 1: a*x + b*y + c = 0,

   Line 2: p*x + q*y + r = 0,

   where a, b, c, p, q, r are all integers.

   The program should check if the two lines are parallel. If the lines are not parallel, calculate and display their point of intersection.

Menu Driven
Control Statement

श्री हर्ष अत्री

Switch-Case statement

- This is very useful to evaluate multiple conditions.
- The switch block defines an expression (or condition) and case has a block of statements, based on the result of expression, corresponding case block gets executed.
- A switch can have any number of cases; however there should be only one default handler.
- The switch statement is often faster than nested if...else (not always). Also, the syntax of switch statement is cleaner and easy to understand.
- When a case constant is found that matches the switch expression, control of the program passes to the block of code associated with that case.

Rules for switch statement in C language

1. The switch expression must be of an integer or character type.
2. The case value must be an integer or character constant.
3. The case value can be used only inside the switch statement.
4. The break statement in switch case is not must. It is optional. *If there is no break statement found in the case, all the cases will be executed present after the matched case.* It is known as fall through the state of C switch statement.
5. To execute only one case from multiple cases, break statement has to be written in every case.

Switch-Case statement

Syntax of switch...case

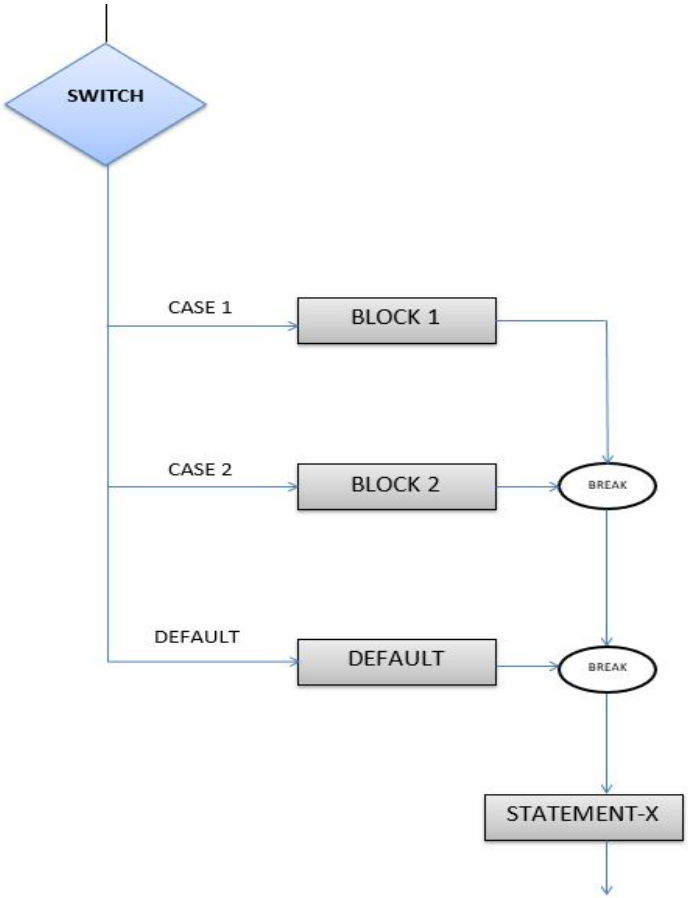
```
switch n                    // where n is case number
{       case 1:     // code to be executed if n is equal to constant1;
        break;
        case 2:     // code to be executed if n is equal to constant2;
        break;
        default:    // code to be executed if n doesn't match any constant
}
```

Example: Assume that there are following variables.

```
int x,y,z;
char a,b;
float f;
```

Valid Switch	Invalid Switch	Valid Case	Invalid Case
switch(x)	switch(f)	case 3:	case 2.5:
switch(x > y)	switch(x + 2.5)	case 'a':	case x:
switch(a + b - 2)		case 1 + 2:	case x + 2:
switch(func (x, y))		case 'x' > 'y':	case 1, 2, 3:

Switch-Case Flowchart



Switch-Case statement

```
main()
{
    int data;
    printf("Enter value -1 or 0 or 1\n");
    scanf("%d",&data);
    switch(data)
    // Please change this part to if-else statement
    {
        case -1:
        printf("This is negative");
        break;
        case 0:
        printf("This is zero");
        break;
        case 1:
        printf("This is positive");
        break;
        default:
        printf("You entered wrong value");
    }
}
```

Difference between if-else and switch-case

	If-else	Switch
Definition	Depending on the condition in the 'if' statement, 'if' and 'else' blocks are executed.	The user will decide which statement is to be executed.
Expression	It contains either logical or equality expression.	It contains a single expression which can be either a character or integer variable.
Evaluation	It evaluates all types of data, such as integer, floating-point, character or Boolean.	It evaluates either an integer, or character.
Sequence of execution	First, the condition is checked. If the condition is true then 'if' block is executed otherwise 'else' block	It executes one case after another till the break keyword is not found, or the default statement is executed.
Default execution	If the condition is not true, then by default, else block will be executed.	If the value does not match with any case, then by default, default statement is executed.
Editing	Editing is not easy in the 'if-else' statement.	Cases in a switch statement are easy to maintain and modify. Therefore, we can say that the removal or editing of any case will not interrupt the execution of other cases.
Speed	If there are multiple choices implemented through 'if-else', then the speed of the execution will be slow.	If we have multiple choices then the switch statement is the best option as the speed of the execution will be much higher than 'if-else'.

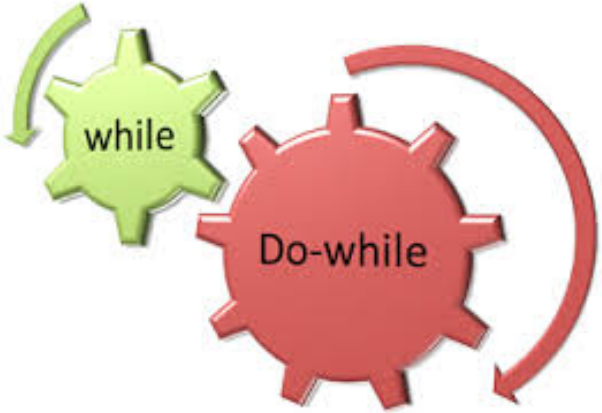
Exercise

Write a Program

1. To create a menu driven program that

   - takes a decimal number as input and converts it to its binary equivalent.
   - takes a binary number as input and converts it to its decimal equivalent and then user should ask again to convert the decimal into octal decimal or hexadecimal number systems.

Iterative Statements



Loop Control Statements

- Loop control statements in C are used to perform looping operations until the given condition is true.
- Control comes out of the loop statements once condition becomes false.

There are 3 types of loop control statements in C language. They are,

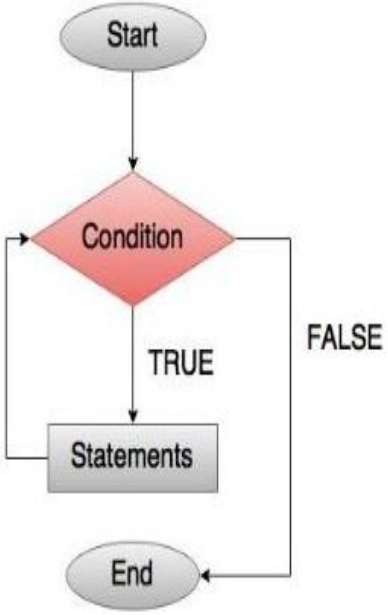
a. while loop
b. do-while loop
c. for loop

While Loop

1. This loop has a mechanism of repeating one or more statements while a particular condition is true.
2. In this type of loop, the condition is generally executed before any of the statements are executed.
3. If the condition is found to be true then only will the statements execute otherwise the condition will be false.
4. While loop will keep on executing as long as the condition is true.

Syntax:

```
statement x;
while (condition)
{
  statement block;
}
statement y;
```



While Loop

Example: Write a program to calculate the sum of first 20 numbers.

```
#include<stdio.h>
main()
{
        int x=0, sum=0;
        while (x<=20)
        {
              sum = sum + x;
              x = x + 1;
        }
        printf("Sum of the first 20 numbers = %d", sum);
}
```

Output:
Sum of the first 20 numbers = 210

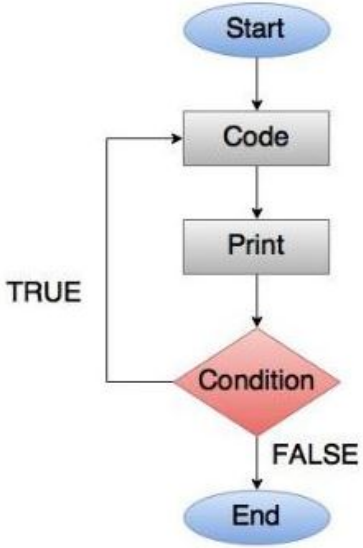
Do-While Loop

It is similar to the while loop with a difference that:

1. In do-while loop control statement, while loop is executed irrespective of the condition for first time.
2. Then 2nd time onwards, loop is executed until condition becomes false.
3. The only difference is that the test condition here is ended at the end of the loop.
4. From this, it is clear that the body of the loop will be executed atleast once.

Syntax:

```
statement x;
do
{
    statement block;
}
while (condition);
statement y;
```



Do-While Loop

Example: Write a program to calculate the average of the first 10 numbers.

```
#include <stdio.h>
main()
{
   int a = 0, sum = 0;
   float avg;
   do
   {
      sum = sum + a;
      a = a + 1;
   }
   while (a<=10);
   avg = sum/10.0;
   printf("The sum of the first 10 numbers are = %d",sum);
   printf("\n The average of the first 10 numbers are = %f",avg);
}
```

Output:

The sum of the first 10 numbers are = 55

The average of the first 10 numbers are = 5.000000

For Loop

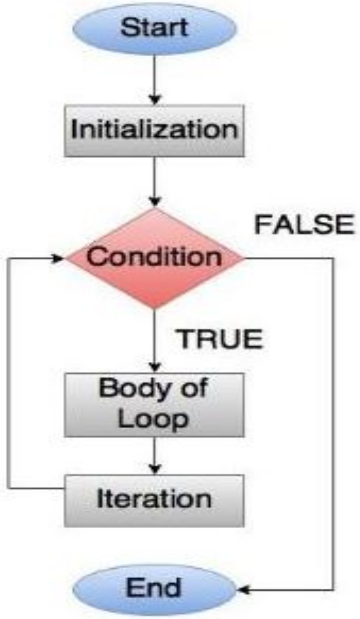
1. It is a determinate or definite loop, as the programmer is aware as to how many times does he want the loop to be repeated.

2. This loop is initialized only once. When the loop is run, with every iteration the value of the loop is updated and the condition is checked.

3. If the condition is true the loop will be executed else the statements are skipped.

Syntax:

```
for (initialization; condition; increment/decrement)
{
    statement block;
}
statement y;
```



For Loop

Example: Write a simple “for loop” program to print the first 5 numbers

```
#include <stdio.h>
main()
{
	int i;
	for (i=1; i<=5 ; i++)
		printf("%d \n", i);
}
```

Output:

1
2
3
4
5

Nested Loop

1. The loops inside the loop are called **Nested loops.** The most popular types of nested loops are **'for'** loops as they are very easy to control.

2. 'For' loop can be used to control the number of times any statement needs to be executed. An outer loop is used for controlling the number of time the loop should be repeated.

3. They should be indented properly so that the reader can easily determine what statements are written within each of them.

Example: Write a program to print the following pattern:

```
#include <stdio.h>
void main()
{
    int i,j;
    for (i=1; i<=5; i++)
    {
        for(j=1; j<=i; j++)
        printf("*");
        printf("\n");
    }
}
```

```
*
**
***
****
*****
```

Exercise

Write a Program

1. To display a multiplication table of a given number.

2. To find the sum of first N natural numbers.

3. To count the number of digits in an integer.

4. To display prime, even and odd numbers between given range.

5. To check whether a number is a palindrome.

6. To find whether a number is Armstrong or not.

7. To read two integers x and y from the user and computes the value of x^y (i.e. x raised to the power y). First, implement the computation manually without using any built-in power functions. Then, verify the result by printing the output of the standard library function pow (x, y).

Exercise

Write a Program

8. To print following patterns

```
* * * * *
* * * *
* * *
* *
*
```

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

```
* * * * *
 * * * *
  * * *
   * *
    *
```

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

```
    *
   ***
  *****
 *******
*********
```

9. To print the calendar of a month, given the total number of days and the first day (1: Monday…7: Sunday) of that month. For example, number of days is 31 and the first day is Tuesday, then the output should be as follows:

M	T	W	T	F	S	S
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31			

10. To display Pascal's triangle. Pascal's triangle represents the binomial coefficients. The first few rows of Pascal's triangle are displayed below. Observe how a row is related to the row above it

```
        1
       1 1
      1 2 1
     1 3 3 1
    1 4 6 4 1
   1 5 10 10 5 1
  1 6 15 20 15 6 1
```

Execution Sequence

goto statement

goto statement

- The “goto” statement is a type of jump statement used to change the flow of execution from one block of code to another.
- The “goto” statement can skip over certain codes or repeat a block by returning to a previously labelled block.
- The concept of **label** is used in this statement to tell the program control where to go. The jump in the program that **goto** take is within the same function.

There are two approaches for using “goto”

Approach 1:
*where the **label** is declared **before** the **“goto”** statement.*

```
//code
label 1;

Statement 1
Statement 2
goto label 1:

Statement 3
Statement 4
//code
```

Approach 2:
*where the **label** is declared **after** the **“goto”** statement.*

```
//code
goto label 1;

Statement 1
Statement 2
label 1:

Statement 3
Statement 4
//code
```

goto statement

Program to demonstrate **goto** statement

```c
#include <stdio.h>
main() {
    int num;
    printf("Enter any number:");
    scanf("%d",&num);

    if (num % 2 == 0) {
        goto even_label;
    } else {
        goto odd_label;
    }

even_label:
    printf("%d is an even number.\n", num);
    goto end_program;

odd_label:
    printf("%d is an odd number.\n", num);
    goto end_program;

end_program:
    printf("Program finished.\n");
}
```

```
Enter any number:24
24 is an even number.
Program finished.
```

```
Enter any number:13
13 is an odd number.
Program finished.
```

break and continue statement

break statement

The break statement **ends the loop immediately** when it is encountered.
The break statement is almost always used with if...else statement inside the loop.

working of break statement in while loop, do-while loop and for loop

```
while (testExpression) {
    // codes
    if (condition to break) {
        break;
    }
    // codes
}
```

```
do {
    // codes
    if (condition to break) {
        break;
    }
    // codes
}
while (testExpression);
```

```
for (init; testExpression; update) {
    // codes
    if (condition to break) {
        break;
    }
    // codes
}
```

break statement

Programmer can use break statements in two ways in C:

1. **break statement in switch case:**

The break statement is used inside a switch statement to terminate the execution of the enclosing switch statement and exit the block of code.
When a break statement is encountered inside a switch block, the control flow is transferred to the end of the switch block and the remaining case statements are skipped.

```
switch(ch)
{  // switch start

case 'A':
{
printf("Enter the two numbers: ");
scanf("%d%d",&a,&b);
sum = a+b;
printf("Addition of %d and %d is %d", a,b,sum);
break;
}
case 'B':
{
printf("Enter the two numbers: ");
scanf("%d%d",&a,&b);
sub = a-b;
printf("Subtraction of %d and %d is %d", a,b,sub);
break;
}
default:
{
printf("You have entered wrong choice");
}
} // switch close
```

break statement

Programmer can use break statements in two ways in C:

2. break statement in loops:

The break statement can be used anywhere inside the **loop's body** of *while loop, do-while loop and for loop.* When executed, it immediately terminates the loop and control is transferred to the statement following the loop.

Demonstration of break statement in while loop

```c
main()
{
    int i;
    printf("Enter the value of i: ");
    scanf("%d",&i);
    while(i<6)
    {
        if (i==5)
        {
            printf("Value of i is %d\n",i);
            break;
        }
        else
        {
            printf("Value of i is %d\n",i);
            i++;
        }
    }
}
```

Terminates the loop when i == 5

Output 1

```
Enter the value of i: 5
Value of i is 5
```

Output 2

```
Enter the value of i: 1
Value of i is 1
Value of i is 2
Value of i is 3
Value of i is 4
Value of i is 5
```

Terminates the loop when i == 5

break statement

Programmer can use break statements in two ways in C:

2. break statement in loops:

The break statement can be used anywhere inside the **loop's body** of *while loop, do-while loop and for loop.* When executed, it immediately terminates the loop and control is transferred to the statement following the loop.

Demonstration of break statement in for loop

```c
main()
{
    int i,a;
    printf("Enter the value of a: ");
    scanf("%d",&a);
    for(i=a;i<6;i++)
    {
        if (i==5)
        {
            printf("Value of i is %d\n",i);
            break;
        }
        else
        {
            printf("Value of i is %d\n",i);
        }
    }
}
```

Terminates the loop when i == 5

Output 1

```
Enter the value of i: 5
Value of i is 5
```

Output 2

```
Enter the value of a: 1
Value of i is 1
Value of i is 2
Value of i is 3
Value of i is 4
Value of i is 5
```

Terminates the loop when i == 5

continue statement

The continue statement is used to skip the current iteration of any loop and bring the control to the beginning of the iteration. The continue statement is almost always used with the if...else statement.

working of continue statement in while loop, do-while loop and for loop

```
while (testExpression) {
    // codes
    if (testExpression) {
        continue;
    }
    // codes
}
```

```
do {
    // codes
    if (testExpression)
        continue;
    }
    // codes
}
while (testExpression)
```

```
for (init; testExpression; update) {
    // codes
    if (testExpression) {
        continue;
    }
    // codes
}
```

continue statement

Demonstration of continue statement

Use of continue statement in while loop

```
main()
{
    int i=1;
    while(i<10)
    {
        if(i==5)
        {
            i++;
            continue;
        }
        printf("i = %d\n",i);
        i++;
    }
}
```

Use of continue statement in for loop

```
main()
{
    int i;
    for(i=1;i<10;i++)
    {
        if(i==5)
        {
            continue;
        }
        printf("i = %d\n",i);
    }
}
```

Output

Skipped the printing when i==5

```
i = 1
i = 2
i = 3
i = 4
i = 6
i = 7
i = 8
i = 9
```

Difference between break and continue

Parameters	Break Statement in C	Continue Statement in C
Purpose	Exits the current loop	Skips the current iteration
Applicability	Used within loops (for, while, do-while)	Used within loops (for, while, do-while)
Effect	Terminates the loop prematurely and continues with the next statement after the loop	Immediately moves to the next iteration of the loop, skipping the remaining code in the current iteration
Loop Control	Affects the entire loop	Affects only the current iteration of the loop
Typical Use Cases	When a specific condition is met, and you want to exit the loop	When you want to skip the current iteration of the loop based on a certain condition and continue with the next iteration

Example	for (int i = 1; i <= 10; i++)
	{
	if (i == 5)
	{
	break;
	}
	printf("%d\n", i); }
		for (int i = 1; i <= 10; i++)
		{
		if (i == 5)
		{
		continue;
		}
		printf("%d\n", i); }

Exercise

Write a Program

1. To use break statement to terminate a loop when a specific number is encountered.
2. To demonstrate the use of continue:
   - to skip even numbers while printing 1 to 10.
   - to skip printing multiples of 3
3. To use **goto** to jump back and repeat input if the user enters a negative number.
4. To use nested loops and break to exit both loops when a condition is satisfied.

References

Web Links:

- https://www.journaldev.com/27602/operators-in-c
- https://www.geeksforgeeks.org/conditional-or-ternary-operator-in-c-c/
- https://fresh2refresh.com/c-programming/c-operators-expressions/c-conditional-operators/
- https://www.programiz.com/c-programming/c-break-continue-statement
- https://www.scholarhat.com/tutorial/c/break-continue-statement-in-c
- https://www.shiksha.com/online-courses/articles/goto-statement-in-c/